

Atividade Prática: Prevendo Sobreviventes do Titanic com Classificação

Objetivo

Você vai utilizar técnicas de classificação para prever se um passageiro do Titanic sobreviveu ou não.

Trabalharemos com o dataset `titanic.csv`, que contém informações reais dos passageiros.

Nesta atividade guiada, você aprenderá a:

- Limpar e preparar dados reais (remover colunas inúteis, tratar valores ausentes).
- Aplicar One-Hot Encoding para transformar variáveis categóricas em números.
- Treinar dois modelos de classificação:
 - Primeiro modelo: você escolhe entre Árvore de Decisão ou KNN (código fornecido).
 - Segundo modelo: você escolhe outro algoritmo da lista do Scikit-learn e implementa por conta própria.
- Avaliar os modelos com métricas como acurácia, matriz de confusão, precisão, recall e F1-score.
- Obter probabilidades de sobrevivência usando `predict_proba`.
- Responder perguntas finais comparando os modelos e analisando os dados.

Ao final, você deverá entregar um notebook com todo o código e as respostas.

1. Conceitos importantes que vamos usar

- Classificação: problema onde a variável alvo é uma categoria (no caso, 0 = não sobreviveu, 1 = sobreviveu).
 - One-Hot Encoding: técnica para converter variáveis categóricas (como sexo ou porto de embarque) em colunas binárias. Por exemplo, a coluna `Sex` com valores 'male' e 'female' vira duas colunas: `Sex_male` (1 se masculino, 0 caso contrário) e `Sex_female` (1 se feminino, 0 caso contrário). Isso é necessário porque algoritmos de machine learning trabalham apenas com números. No nosso dataset, `Sex` já vem como 0 e 1, mas ainda assim é categórica – vamos aplicar One-Hot Encoding para garantir que o modelo não interprete como ordem.
 - Valores ausentes: precisam ser tratados (preenchidos ou removidos).
 - Matriz de confusão: tabela que mostra acertos e erros: Verdadeiros Positivos (VP), Falsos Positivos (FP), Verdadeiros Negativos (VN), Falsos Negativos (FN).
 - Acurácia: $(VP + VN) / \text{total}$.
 - Precisão: $VP / (VP + FP)$ – dentre os que o modelo disse que sobreviveram, quantos realmente sobreviveram.
 - Recall: $VP / (VP + FN)$ – dentre os verdadeiros sobreviventes, quantos o modelo acertou.
 - F1-score: média harmônica entre precisão e recall.
 - Probabilidades: alguns modelos (como Árvore, KNN, Regressão Logística) podem fornecer a probabilidade de cada classe através do método `predict_proba()`. Isso será útil para entender o quão confiante o modelo está.
-

2. O Dataset

O arquivo `titanic.csv` contém as seguintes colunas (algumas são irrelevantes e serão removidas):

Coluna	Descrição
--------	-----------

Passengerid	Identificador único do passageiro (inútil para predição)
Age	Idade
Fare	Tarifa paga
Sex	Sexo (0 = feminino, 1 = masculino) – já numérico?
sibsp	Número de cônjuges/irmãos a bordo
Parch	Número de pais/filhos a bordo
Pclass	Classe do bilhete (1, 2, 3)
Embarked	Porto de embarque (0 = Cherbourg, 1 = Queenstown, 2 = Southampton?) – cuidado: os dados podem ter números, mas é categórico.
Survived	Variável alvo: 1 se sobreviveu, 0 caso contrário (vamos renomear para Survived)

Além disso, há várias colunas chamadas **zero** que são apenas placeholders (todas com valor zero). Elas devem ser removidas.

3. Passo a passo guiado

3.1. Carregar e explorar os dados

```
python
import pandas as pd
import numpy as np

# Carregar o arquivo
df = pd.read_csv('titanic.csv')

# Renomear a coluna alvo (de '2urvived' para 'Survived')
df.rename(columns={'2urvived': 'Survived'}, inplace=True)

# Ver as primeiras linhas
print(df.head())

# Informações gerais (tipos, valores não nulos)
print(df.info())

# Estatísticas descritivas
print(df.describe())

# Verificar valores ausentes
print(df.isnull().sum())
```

3.2. Limpeza inicial

Remova colunas que não ajudam na predição:

- Colunas chamadas zero (todas constantes e iguais a zero).
- Passengerid (é só um identificador).

```
python
# Listar colunas que começam com 'zero'
zero_cols = [col for col in df.columns if col.startswith('zero')]
df.drop(columns=zero_cols, inplace=True)

# Remover Passengerid
df.drop(columns=['Passengerid'], inplace=True)

# Verifique as colunas restantes
print(df.columns)
```

3.3. Tratar valores ausentes

Verifique quais colunas têm valores faltantes. Provavelmente `Age` e `Embarked`.

- Para `Age`, vamos preencher com a mediana das idades (a mediana é menos sensível a valores extremos que a média).
- Para `Embarked`, vamos verificar quantos valores faltam. Se forem poucos, podemos preencher com o valor mais frequente (moda) ou remover as linhas. Vamos preencher com a moda.

```
python
# Preencher Age com a mediana
df['Age'].fillna(df['Age'].median(), inplace=True)

# Verificar Embarked
print(df['Embarked'].value_counts())
# Se houver missing, preencha com o valor mais frequente
df['Embarked'].fillna(df['Embarked'].mode()[0], inplace=True)
```

3.4. One-Hot Encoding – O que é e como aplicar

O que é One-Hot Encoding?

Variáveis categóricas são aquelas que representam categorias (ex.: sexo, porto de embarque). Se deixarmos os números como estão (ex.: 0, 1, 2), o modelo pode interpretar que existe uma ordem (por exemplo, que 2 é maior que 1), o que não faz sentido. One-Hot Encoding cria colunas binárias para cada categoria.

Por exemplo, a coluna `Sex` tem valores 0 (feminino) e 1 (masculino). Após o One-Hot Encoding, podemos criar duas colunas: `Sex_female` (1 se feminino, 0 caso contrário) e `Sex_male` (1 se masculino, 0 caso contrário). No entanto, para evitar multicolinearidade, geralmente descartamos uma das colunas (parâmetro `drop_first=True`). Assim, se tivermos apenas a coluna `Sex_male`, quando ela for 0 significa feminino, quando for 1 significa masculino – a informação é preservada.

Como aplicar no Python:

Usamos a função `pd.get_dummies()`. Vamos aplicá-la às colunas `Sex` e `Embarked`. Também podemos considerar `Pclass` como categórica (embora seja ordinal, muitos modelos tratam bem como numérica, mas para fins didáticos vamos codificá-la também).

```
python
# Aplicar One-Hot Encoding em Sex, Embarked e Pclass
# drop_first=True remove a primeira categoria para evitar redundância
df = pd.get_dummies(df, columns=['Sex', 'Embarked', 'Pclass'], drop_first=True)

# Visualize as colunas resultantes
print(df.head())
```

Agora seu DataFrame está totalmente numérico e pronto para o treinamento. Note que colunas como `Sex_1`, `Embarked_1`, `Embarked_2`, `Pclass_2`, `Pclass_3` foram criadas.

3.5. Separar features e target, e dividir em treino e teste

```
python
from sklearn.model_selection import train_test_split

X = df.drop('Survived', axis=1)
y = df['Survived']

# Dividir em treino (80%) e teste (20%) com stratify para manter a proporção de sobreviventes
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

3.6. Normalização/Padronização (opcional, mas importante para alguns modelos)

Alguns algoritmos (como KNN e SVM) são sensíveis à escala das features. Outros (como árvores) não. Para padronizar, usamos `StandardScaler` que transforma os dados para média 0 e desvio padrão 1.

```
python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Nota: Se você for usar árvore, pode usar os dados originais ou os escalados – ambos funcionam. Vamos manter os dados escalados para facilitar a comparação.

4. Modelo 1 – Escolha entre Árvore de Decisão ou KNN

Agora você deve escolher um dos dois algoritmos abaixo. O código já está pronto; você só precisa executar e entender os resultados.

Opção A: Árvore de Decisão

```
python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Criar e treinar a árvore (limitando a profundidade para evitar overfitting)
tree = DecisionTreeClassifier(max_depth=5, random_state=42)
tree.fit(X_train_scaled, y_train)

# Fazer previsões no teste
y_pred_tree = tree.predict(X_test_scaled)

# Avaliar
print("Acurácia da Árvore:", accuracy_score(y_test, y_pred_tree))
```

```

print("\nRelatório de Classificação:\n", classification_report(y_test,
y_pred_tree))
print("\nMatriz de Confusão:\n", confusion_matrix(y_test, y_pred_tree))

# Importância das features (a árvore calcula automaticamente)
importancias = pd.DataFrame({
    'feature': X.columns,
    'importancia': tree.feature_importances_
}).sort_values('importancia', ascending=False)
print("\nImportância das features:\n", importancias)

```

Opção B: KNN (K-Nearest Neighbors)

```

python
from sklearn.neighbors import KNeighborsClassifier

# Criar e treinar o KNN (vamos usar k=5, mas você pode testar outros valores)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_scaled, y_train)

y_pred_knn = knn.predict(X_test_scaled)

print("Acurácia do KNN:", accuracy_score(y_test, y_pred_knn))
print("\nRelatório de Classificação:\n", classification_report(y_test, y_pred_knn))
print("\nMatriz de Confusão:\n", confusion_matrix(y_test, y_pred_knn))

# O KNN não fornece importância das features diretamente.

```

Interpretando os resultados:

- A acurácia dá uma ideia geral, mas se a base estiver desbalanceada (poucos sobreviventes), o F1-score para a classe 1 é mais importante.
- A matriz de confusão mostra quantos erros o modelo cometeu e de que tipo.
- No relatório de classificação, observe a precisão, recall e f1 para cada classe (0 e 1).

5. Obtendo probabilidades de sobrevivência

Muitos classificadores possuem o método `predict_proba()`, que retorna a probabilidade estimada para cada classe. Por exemplo, para a primeira amostra do teste, podemos ver a chance de sobreviver.

```
python
# Para a árvore (ou KNN)
probabilidades = tree.predict_proba(X_test_scaled) # ou knn.predict_proba

# Exemplo: probabilidades para as 5 primeiras amostras
print(probabilidades[:5])
```

Cada linha retorna um par `[probabilidade_classe_0, probabilidade_classe_1]`. A soma é 1.

Você pode usar isso para analisar a confiança do modelo em suas previsões.

6. Modelo 2 – Escolha outro algoritmo de classificação

Agora é sua vez! Escolha um outro classificador dentre as opções abaixo. Você deverá:

- Importar a classe correspondente.
- Criar o modelo (se quiser, ajuste algum parâmetro – consulte a documentação do Scikit-learn).
- Treinar com `X_train_scaled` e `y_train`.
- Prever e avaliar com as mesmas métricas (acurácia, `classification_report`, matriz de confusão).
- Se o modelo tiver `feature_importances_` ou `coef_`, extraia e exiba. Caso contrário, comente.

Modelo	Classe no sklearn	Breve descrição
--------	-------------------	-----------------

Regressão Logística	<code>LogisticRegression</code>	Modelo linear para classificação.
SVM (Support Vector Machine)	<code>SVC</code>	Busca um hiperplano separador. Pode usar diferentes kernels.
Random Forest	<code>RandomForestClassifier</code>	Conjunto de árvores de decisão, geralmente mais robusto.
Gradient Boosting	<code>GradientBoostingClassifier</code>	Constrói árvores sequencialmente corrigindo erros.
MLP (rede neural simples)	<code>MLPClassifier</code>	Rede neural de múltiplas camadas.
Naive Bayes	<code>GaussianNB</code>	Baseado no teorema de Bayes, assume independência das features.
Árvore de Decisão (se não usou no primeiro)	<code>DecisionTreeClassifier</code>	–
KNN (se não usou no primeiro)	<code>KNeighborsClassifier</code>	–

Importante: Não vale repetir o mesmo algoritmo do primeiro modelo.

Exemplo de implementação para Random Forest:

```
python
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_train_scaled, y_train)

y_pred_rf = rf.predict(X_test_scaled)

print("Acurácia Random Forest:", accuracy_score(y_test, y_pred_rf))
print("\nRelatório:\n", classification_report(y_test, y_pred_rf))
print("\nMatriz de Confusão:\n", confusion_matrix(y_test, y_pred_rf))

# Importância das features
importancias_rf = pd.DataFrame({
    'feature': X.columns,
    'importancia': rf.feature_importances_
}).sort_values('importancia', ascending=False)
print(importancias_rf)
```

Agora faça o mesmo para o algoritmo que você escolheu.

7. Comparação e perguntas finais

Crie uma tabela resumo com os resultados dos dois modelos (inclua acurácia, precisão, recall e F1 para a classe 1 – sobreviventes). Você pode exibir no notebook como markdown ou simplesmente printar.

Em seguida, responda às perguntas abaixo em células de texto (Markdown).

Perguntas

1. Qual modelo apresentou o melhor desempenho? Considere a acurácia e, principalmente, o F1-score para a classe dos sobreviventes (já que a base pode estar desbalanceada). Justifique.
 2. Quais foram as 3 características mais importantes para a sobrevivência, de acordo com o melhor modelo? (Se o melhor modelo não fornecer importância, use o que fornecer, ou comente essa limitação.)
 3. Analise a matriz de confusão do melhor modelo. O modelo errou mais classificando sobreviventes como mortos ou mortos como sobreviventes? O que isso pode indicar? (Por exemplo, se o modelo erra mais falsos negativos, pode ser mais conservador em prever sobrevivência.)
 4. Utilizando apenas pandas (sem modelos), calcule a taxa de sobrevivência (porcentagem) entre homens e mulheres no dataset.
Dica: Agrupe por 'Sex' (lembre-se de que você aplicou One-Hot Encoding; a coluna que indica masculino é `Sex_1` – ela vale 1 para homens, 0 para mulheres). Calcule a média da coluna `Survived` para cada grupo e multiplique por 100.
 - Qual é a porcentagem de mulheres que sobreviveram? E de homens?
 - Esses números são consistentes com o que você esperava da história do Titanic? Por quê?
 5. Os resultados dos dois modelos foram muito diferentes? O que isso pode sugerir sobre a natureza dos dados (linearidade, interações, etc.)?
-

8. Entrega

- Um notebook (.ipynb) ou script Python bem comentado, contendo todas as etapas, os resultados e as respostas às perguntas.
 - Nome do arquivo: `seunome_titanic_classificacao.ipynb`
-

Dicas finais

- Use `random_state=42` sempre que possível para garantir reproducibilidade.

- Consulte a documentação do Scikit-learn para entender os parâmetros dos modelos: <https://scikit-learn.org/>
- Se tiver dúvidas sobre as métricas, pergunte ao professor ou pesquise.
- Divirta-se explorando os dados e os modelos! 🚢

Bom trabalho!